

Сборка файла по шагам — учебное пособие

Это третья, самая подробная версия руководства по сборке файла и разрешению коллизий — «для учащихся техникума»: каждый термин вводится с нуля, каждый шаг алгоритма разобран на числах, с таблицами и трассами. Формальная монография — [AssemblyGuide.Academic.ru.md](#), короткий инженерный пересказ — [AssemblyGuide.Engineer.ru.md](#) (нумерация разделов у них совпадает между собой; здесь своя, «уроковая»). Общий контекст — формат пакета, хеши, накопление — в [AlgorithmGuide.ru.md](#).

Прочитав это пособие, вы сможете своими словами объяснить: откуда берутся версии секторов, почему сборка проверяется одним финальным хешем, как выбирается «кандидат №1», что такое точка ветвления, как работает полный перебор («одометр»), эвристическая прокрутка и подбор подмножества томов, и почему подделка в принципе не может превратиться в собранный файл.

1. Словарь: слова, без которых дальше никуда

Термин	Что это на самом деле
Сектор (том)	Один пронумерованный кусок данных длиной ровно 64 байта. Номера: <code>0 ... T-1</code> . Первые <code>N</code> номеров — data-тома (куски файла), последние <code>M</code> — ECC-тома (избыточность для восстановления потерянного).
<code>N</code>	Число data-томов. Не хранится в заголовке, а вычисляется: <code>N = ceil(FileSize / 64)</code> , минимум 1.
<code>M</code>	Число ECC-томов (из заголовка). Может быть 0.
<code>T</code>	<code>T = N + M</code> — общее число томов. Потолок — 65 535.
Payload	Сами 64 байта сектора: либо кусок файла, либо ECC-данные.
Заголовок	51-байтная «шапка» файла: имя, размер, SHA-256 файла, <code>M</code> .
<code>H3</code>	SHA-256 содержимого исходного файла (32 байта, поле заголовка). Главный судья сборки.
<code>H5</code>	24-байтный хеш заголовка; сид, из которого считаются хеши секторов.
Версия	Конкретный вариант payload для одного номера сектора.
Счётчик подтверждений	Сколько раз эта версия приходила из потока. У любой версии ≥ 1 .
Коллизия	У одного номера больше одной версии.
Равновероятная голова	Начало списка версий: все лидеры с одинаковым максимальным счётчиком.
Точка ветвления	Сектор, у которого в голове ≥ 2 версий. Именно его «спорные» варианты перебираются.
Выборка (<code>selecte d</code>)	Набор «по одному payload на каждый номер от 0 до T-1». Один кандидат на готовый файл.
Попытка	Проверка одной выборки: прямая сборка, при неудаче — RS-восстановление, затем финальный хеш.
Слот приёма	Накопитель всего, что принято про один файл: заголовок + списки версий. Класс <code>ReceptionSlot</code> .
Одометр	Алгоритм полного перебора комбинаций — как счётчик километража: младший разряд крутится быстрее.
Прокрутка	Эвристический обход: все спорные списки одновременно сдвигаются на один шаг.
Маска исключения	Набор номеров томов, которые попытка стадии 3 сознательно считает стёртыми.

2. Что лежит на столе у сборщика

Декодер уже отсканировал поток и разложил принятое по полочкам. Напомним, как выглядит один пакет — 75 байт:

Поле	Размер, байт	Смещение	Смысл
Номер сектора	2	0	0 ... T-1
Payload	64	2	кусоч файла (data) или избыточность (ECC)
Хеш сектора (D3)	9	66	усечённый SHA-256, 72 бита — быстрая проверка «не побился ли пакет»

Заголовок ездит по потоку отдельным пакетом: 51 байт полей + 24 байта H5 — те же 75 байт. Поля заголовка:

Поле	Размер, байт	Смещение	Смысл
Имя файла	14	0	фиксированная длина, добивается нулями
Размер файла	3	14	little-endian; максимум 16 777 215 байт (~16 МиБ)
SHA-256 файла (H3)	32	17	арбитр подлинности
Число ECC-томов M	2	49	little-endian

В текстовом потоке каждый 75-байтный пакет ещё и закодирован Base64 — ровно в 100 символов. Итак, к моменту сборки у нас есть:

- заголовок → значит, известны N, M, T, FileSize и эталонный хеш H3;
- куча принятых секторов, разложенных по номерам. У одного номера может лежать несколько разных payload — это версии, у каждой свой счётчик.

Всё дальнейшее — ответ на вопрос: какие именно версии взять, чтобы файл собрался.

3. Почему проверка одна и финальная

Формула сборки предельно проста:

```
буфер = payload[0] || payload[1] || ... || payload[N-1]           // N × 64 байт
результат = буфер, обрезанный до FileSize байт
если SHA-256(результат) == H3 → файл готов
иначе → эта выборка неверна, пробуем другую
```

Три следствия, которые стоит выучить как таблицу умножения:

- Проверка не зависит от процесса приёма.** Ни счётчики, ни порядок прихода пакетов, ни решения эвристик не участвуют в формуле. Судья один — SHA-256.

- Результат бинарный.** Либо файл, бит-в-бит равный исходнику (доказательство — совпадение хеша), либо честный отказ `null`. Состояния «почти собрали, выдадим как есть» не существует в принципе.
- Обмануть финальный хеш нельзя.** Для этого нужен прообраз SHA-256 — на сегодня практически нереально. Максимум вреда от злоумышленника — заставить нас перебирать версии и потратить время.

4. Как накапливаются версии: `AddSector` по шагам

Сердце накопителя — словарь «номер сектора → список версий», причём список **всегда** отсортирован по убыванию счётчика (это инвариант, он проверяется в тестах).

Приём одной копии сектора (`AddSector(sectorNum, payload)`) выглядит так:

- Две проверки на входе.** Номер в диапазоне `0 ... T-1`? Payload ровно 64 байта? Нет — копия молча отбрасывается, возвращается `false`.
- Поиск существующей версии.** Новый payload побайтово сравнивается с каждой версией списка.
- Нашли такую же** → её счётчик `+1` (с насыщением: выше `int.MaxValue` не растёт). После этого версия «всплывает» вверх — но только **строго выше** версий с меньшим счётчиком. Через версии с таким же счётчиком она **не переставляется**: порядок равных неприкосновенен (это важно для прокрутки, раздел 11).
- Не нашли** → в конец списка добавляется новая версия со счётчиком 1. В конце — потому что у всех существующих счётчик уже ≥ 1 , и сортировка не нарушается.

Пример. Пусть сектор № 1 приходит в порядке: `P, P, Q, P, Q`. Список живёт так:

Пришла копия	Действие	Список после
<code>P</code>	новой версии нет → добавить, счётчик 1	<code>[P×1]</code>
<code>P</code>	совпала с <code>P</code> → счётчик 2	<code>[P×2]</code>
<code>Q</code>	не совпала ни с кем → новая версия в конец	<code>[P×2, Q×1]</code>
<code>P</code>	совпала → счётчик 3	<code>[P×3, Q×1]</code>
<code>Q</code>	совпала → счётчик 2; всплывать нечего: впереди <code>P×3</code>	<code>[P×3, Q×2]</code>

А вот пример всплытия. Было `[X×3, P×2]`:

Пришла копия	Список после	Почему
<code>P</code>	<code>[X×3, P×3]</code>	счётчики сравнялись, но равных не перепрыгиваем
<code>P</code>	<code>[P×4, X×3]</code>	<code>4 > 3</code> — версия всплыла на первое место

Обратите внимание: обычные повторы (тот же payload ещё раз) — это **не** коллизия, а укрепление истины. Коллизия — это именно разные payload у одного номера.

5. Откуда вообще берутся разные версии одного номера

1. **Случайная коллизия 72-битного хеша сектора.** Вероятность исчезающе мала, но теоретически «чужой» пакет может пройти быструю проверку D3.
2. **Подделка.** H5 передаётся открыто, поэтому любой, кто видит поток, может нагенерировать «валидные» секторы с любым содержимым. Весь механизм версий и перебора заточен именно под этот случай.

Против подделки работают три слоя: счётчики (повторы усиливают правду), перебор равновероятных кандидатов и финальный хеш (см. раздел 3).

6. Кандидат №1: наиболее подтверждённые версии

Первая выборка, которую сборка пробует, собирается тривиально: **первый элемент каждого списка** — то есть самые подтверждённые версии. Неспорные секторы (одна версия или явный лидер) вообще не создают выбора: их вклад в любую попытку одинаков.

Пример. T = 4, списки:

```
№0: [P0×5]           — одна версия, выбора нет
№1: [P1×3, Q1×3]      — ничья на вершине: выбор есть
№2: [P2×7, R2×2]      — явный лидер: выбора нет
№3: [P3×1, Q3×1]      — ничья: выбор есть
```

Кандидат №1: [P0, P1, P2, P3] — везде первые элементы.

7. Простейший случай: спора нет

Если ни у одного сектора нет равновероятной головы из ≥ 2 версий, то и перебирать нечего: выборка определена однозначно. Сборка делает ровно одну попытку (прямая сборка, при неудаче — RS-восстановление, раздел 14), при успехе сообщает `AssemblyFinished` и возвращает файл. Большинство честных приёмов заканчивается здесь, на первом же шаге.

8. Спорные секторы: точки ветвления

Если спора есть, сборщик находит все **точки ветвления**. Для каждого списка с ≥ 2 версиями он смотрит на «голову»: сколько первых версий имеют тот же счётчик, что и лидер. Это число — размер головы.

Список версий	Размер головы	Точка ветвления?
$[P \times 3, Q \times 1]$	1	нет — лидер единоличный
$[P \times 3, Q \times 3]$	2	да, перебираем P и Q
$[P \times 2, Q \times 2, R \times 1]$	2	да, перебираем P и Q ; R не участвует
$[P \times 1, Q \times 1, R \times 1]$	3	да, перебираем все три

Важно проговорить: **в поиске участвуют только версии из голов**. Версии-меньшинства ($R \times 1$ в третьей строке) в текущем вызове сборки не проверяются вовсе. Это осознанный компромисс: пространство поиска держится маленьким, а если правда придёт ещё раз — её счётчик вырастет, и она сама попадёт в голову.

9. Сколько всего комбинаций

Общее число комбинаций — произведение размеров голов: $C = h_1 \times h_2 \times \dots \times h_k$.

- одна точка с головой 2 → $C = 2$;
- две точки $3 \times 2 \rightarrow C = 6$;
- три точки $2 \times 2 \times 2 \rightarrow C = 8$;
- пять точек $10 \times 10 \times 10 \times 10 \times 10 \rightarrow C = 100\,000$ — ровно на грани лимита (раздел 10).

Если произведение переполняет 64-битное число, оно зажимается в `long.MaxValue`, а в сообщении прогресса печатается заглушка «больше максимума». Исключений переполнение не вызывает.

10. Развилка: полный перебор или прокрутка

Перебор всех C комбинаций — самый надёжный, но и самый дорогой путь: каждая попытка — это склейка буфера и подсчёт SHA-256. Поэтому перед стартом сборщик делает простую арифметику:

- Пробует кандидата №1** (нулевую комбинацию) и **замеряет время** этой попытки — t_0 , с точностью до тиков `Stopwatch`. Если успех — вопрос закрыт, никакой поиск не нужен.
- Оценивает полный проход:** $\text{оценка} = t_0 \times C$.
- Решает по двум условиям сразу:**

Условие	Порог по умолчанию
комбинаций не слишком много	$C \leq 100\,000$
оценка времени в бюджете	$t_0 \times C \leq 30\text{ с}$

Оба выполнены → **полный перебор** (одометр). Хоть одно нет → **эвристическая прокрутка**.

Числовой пример. Первая попытка заняла 5 мс, комбинаций 4 000 → оценка $5 \text{ мс} \times 4000 = 20 \text{ с}$ $\leq 30 \text{ с}$ → перебор. А если попытка стоит 10 мс и комбинаций 6 000 → оценка $60 \text{ с} > 30 \text{ с}$ → прокрутка: она не обязана обойти всё, зато дёшево проверяет «самые правдоподобные диагонали».

Бюджет времени — **мягкий**: он проверяется между попытками, а не прерывает попытку на середине. Плюс между попытками проверяется отмена (кнопка «Стоп» в интерфейсе).

11. Прокрутка: эвристика «все крутим на один шаг»

Сначала — эвристика, потому что она проще. После каждой неудачной попытки каждый спорный список сдвигается на один элемент: **первая версия головы переносится в конец головы**. Версии-меньшинства, лежащие ниже, не трогаются — поэтому сортировка списка по убыванию счётчика не нарушается никогда.

Классический пример: головы $A/B/C$ (размер 3) и X/Y (размер 2). Старт — кандидат №1. Далее каждая строка = «сдвинули списки → взяли первые элементы → проверили»:

Состояние	Список 1	Список 2	Выборка
о (старт)	A B C	X Y	A + X
1	B C A	Y X	B + Y
2	C A B	X Y	C + X
3	A B C	Y X	A + Y
4	B C A	X Y	B + X
5	C A B	Y X	C + Y

Все списки крутятся синхронно — поэтому проверяются «диагонали», а не всё произведение. Сколько уникальных состояний? Столько, через сколько шагов **все** списки одновременно вернутся в исходный порядок, — это наименьшее общее кратное (НОК) размеров голов: $\text{НОК}(3, 2) = 6$. В примере выше прокрутка обошла все 6 комбинаций — потому что 3 и 2 взаимно просты, и НОК совпал с произведением.

А вот головы $2 \times 2 \times 2$: произведение $C = 8$, но $\text{НОК}(2, 2, 2) = 2$. Прокрутка проверит только $A+X+P$ и $B+Y+Q$; шесть «смешанных» комбинаций (например, $A+Y+P$) она не увидит никогда. Если истина среди них — будет честный отказ. Это плата за скорость: полный проход стоил бы вчетверо дороже.

Число состояний ограничено сверху лимитом попыток (по умолчанию 100 000, вместе с исходным состоянием). Если НОК больше — прокрутка просто остановится на лимите.

12. Полный перебор: одометр

Одометр устроен как счётчик километража: у каждой точки ветвления своя «цифра», младшая — справа. Шаг вперёд: младшая цифра `+1`; если она достигла своего размера головы — обнуляется, а `+1` переносится в соседнюю слева. Когда перенос выходит за старшую цифру — комбинации исчерпаны.

Пример для голов `3 × 2` (старшая — список из трёх, младшая — из двух). Нулевая комбинация уже проверена как «кандидат №1», цикл стартует с первой:

Попытка	Цифры одометра	Выборка
0 (до цикла)	<code>(0, 0)</code>	<code>A + X</code>
1	<code>(0, 1)</code>	<code>A + Y</code>
2	<code>(1, 0)</code>	<code>B + X</code>
3	<code>(1, 1)</code>	<code>B + Y</code>
4	<code>(2, 0)</code>	<code>C + X</code>
5	<code>(2, 1)</code>	<code>C + Y</code>

Шесть попыток — **все** комбинации без пропусков и повторов. Между попытками проверяются отмена и бюджет времени; при исчерпании комбинаций или бюджета возвращается `null`. Сравните с разделом 11: на головах `3 × 2` оба алгоритма проходят одно и то же множество выборок — только одометр «строками», а прокрутка «диагоналями».

13. Тихие повреждения и подбор подмножества томов

Обе стратегии из разделов 11–12 работают, когда поломка **видна**: есть коллизия — есть из чего выбирать. А теперь представьте худшее: сектор № 7 пришёл ровно один раз, номер его цел, 72-битный хеш сектора сошёлся — формально пакет честный. А на самом деле payload побит: носитель «подлечил» байт, или злоумышленник сгенерировал пакет сразу с корректным хешем. Версия одна, головы нет, точки ветвления нет. Прямая сборка и RS используют том № 7 «как есть», финальный хеш не сходится, а перебирать нечего.

Стадия 3 меняет сам вопрос. Вместо «какие версии взять» — «какие тома НЕ брать». Подозрительный том помечается стёртым, как будто он вообще не приходил, и RS восстанавливает его содержимое из оставшихся томов и ЕСС. Восстановили, склеили, проверили финальным хешем. Это и есть **подбор подмножества томов** — последняя линия поиска (прямую сборку и перебор версий будем называть стадиями 1–2).

Кого исключать и в каком порядке, решает планировщик:

1. **База — все коллизионные секторы сразу.** Раз у сектора несколько версий, а предыдущие стадии не разобрались, — не щёлкаем версиями, а стираем весь сектор и восстанавливаем его по ЕСС. Если даже база не помещается в ЕСС-запас, стадия сразу признаёт бессилие (пустой план).

2. **Уровень 1 — по одному.** Каждый присутствующий бесспорный сектор по очереди объявляется стёртым. Порядок — по подозрительности: у кого меньше подтверждений, тот раньше; при равных счётчиках — в случайном порядке (сид случайности берётся из H5, поэтому план одинаков у всех и не зависит от порядка прихода пакетов).
3. **Уровни 2 и 3 — пары и тройки** из 64 самых подозрительных секторов (шорт-лист): вдруг побито сразу два-три.

Числовой пример. $N = 64$ data-тома, $M = 8$ ECC-томов (запас покрывает до 8 потерь). Тихо испорчен только том № 7:

Шаг	Маска исключения	Что делает RS	Итог
стадии 1–2	— (полная карта)	том № 7 считается целым	хеш не сошёлся
стадия 3: база	∅ (коллизий нет)	—	сразу уровни
уровень 1, попытка 1	{0}	восстановил № 0 из остальных	хеш не сошёлся
...	...	...	...
уровень 1	{7}	восстановил № 7 — истину	хеш сошёлся

Каждая попытка стадии 3 — полноценное RS-восстановление: сначала дорогая подготовка (гауссова инверсия, $\sim E^2 \cdot N$ операций, где E — число стёртых data-томов), затем быстрое восстановление. Поэтому у стадии два отдельных ограничителя: **кап на E** (по умолчанию 32 — при большем одна попытка становится слишком дорогой) и пара «число попыток / бюджет времени» (100 000 / 30 с). В прогрессе стадия видна как «Подбор томов, попытка: k / оценка»; отмена проверяется между попытками.

Заметьте: исключить можно и ECC-том — подозрительность не зависит от роли. Правда, каждое исключённое «избыточное» уменьшает запас восстановления, и лимит осуществимости маски это учитывает.

14. Одна попытка под микроскопом

И перебор, и прокрутка на каждом шаге делают одно и то же — проверяют очередную выборку. Внутри попытки два этапа, строго по порядку.

Этап 1 — прямая сборка. Все первые N номеров выборки заполнены (не `null`)? Если хоть один data-том отсутствует — попытка сразу провалена. Иначе: буфер $N \times 64$ байт, копируем payload подряд, обрезаем до `FileSize`, считаем SHA-256, сравниваем с `H3` (подробно — раздел 15). Совпал — файл готов.

Этап 2 — RS-восстановление (только если задан RS-адаптер и этап 1 провалился):

- В заголовке заявлено $M > 0$? Нет ($M = 0$) — попытка провалена окончательно.
- Строим карту присутствия по всем T номерам и считаем, сколько томов (data + ECC) у нас есть. Их должно быть **не меньше N** — иначе восстанавливать не из чего, провал.
- Отдаём коду Рида — Соломона принятые тома и карту; он возвращает N восстановленных кусков. Если декодер вернул не N кусков или хоть один не 64 байта — провал.

4. Куски склеиваются в буфер, обрезаются, проверяются по `h3` — как на этапе 1.

Почему прямая сборка первая? Она дешевле (нет математики Рида — Соломона) и в честном полном приёме почти всегда сразу успешна. RS — запасной путь для реально потерянных томов.

15. Финальный арбитраж и гигиена памяти: `TrimAndVerif`

У

Последний бастион — маленькая, но принципиальная функция:

1. Буфер `N × 64` байт обрезается до `FileSize` (если файл не кратен 64, последний том содержит «хвост» из лишних байтов — они отбрасываются).
2. Считается SHA-256 результата и сравнивается с `h3` из заголовка.
3. **Совпал** → результат возвращается вверх: файл собран.
4. **Не совпал** → оба буфера (полный и обрезанный, если это разные массивы) **заполняются нулями** и возвращается `null`. Зачем обнулять? Чтобы «почти правильные» данные не остались в памяти и не утекли дальше по ошибке: неправильный кандидат не должен существовать ни в каком виде.

16. Прогресс: что видит пользователь

Прогресс идёт по глобальной шкале 0...100 и троттлится до целых процентов; 100 не показывается, пока файл реально не собран. Внутри поиска счётчик идёт по комбинациям, а не по байтам:

- старт поиска: «Поиск версий секторов: <число комбинаций>»;
- перебор: «Полный перебор: 4231 / 100000»;
- прокрутка: «Прокрутка версий: k / НОК»;
- восстановление: своя фаза RS;
- финал: «Сборка завершена» с процентом 100 — только при успехе.

Отмена (`CancellationToken`) проверяется между попытками и бросает исключение наружу — интерфейс корректно показывает «прервано пользователем», а не «файл не собрался».

17. Приборная панель слота

Всё, что происходит внутри, наблюдается снаружи — этими метриками пользуется интерфейс:

Метра	Что показывает
<code>HeaderReceptionCount</code>	сколько копий заголовка поймано
<code>ReceivedSectorCount</code>	сколько номеров закрыты хотя бы одной версией
<code>ReceivedSectorCopyCount</code>	сколько всего копий секторов принято (сумма всех счётчиков)
<code>CollisionSectorCount</code>	сколько номеров с конкурирующими версиями
<code>Coverage</code>	процент закрытых номеров от <code>T</code>
<code>FormatValidityMap</code>	строка-карта:  принят,  коллизия,  дыра
<code>BuildCollisionMap</code>	словарь «номер → сколько версий»
<code>GetSectorVersions(n)</code>	снимок версий конкретного сектора в порядке предпочтительности

Пример карты для `T = 10`: строка  читается так — 7 номеров закрыты (из них 2 с коллизиями), 3 дыры, покрытие 70%.

18. Пять учебных сценариев на числах

18.1. Чистый приём

Файл 100 байт, без ECC: `N = ceil(100/64) = 2`, `M = 0`, `T = 2`. Оба сектора приняты по одному разу. Точек ветвления нет → одна попытка: буфер 128 байт → обрезка до 100 → SHA-256 совпал с `H3` → готово. Прогресс: сразу `AssemblyFinished`.

18.2. Один спорный сектор

Тот же файл, но сектор № 0 пришёл в двух вариантах: `A×3` и `B×3`. Голова размера 2 → одна точка ветвления, `C = 2`, оценка времени заведомо в бюджете → перебор. Попытка 0 (кандидат №1): `A` — хеш не сошёлся. Попытка 1 (одометр `(0) → (1)`): `B` — сошёлся. Истина найдена максимум со второй попытки.

18.3. Два спорных сектора: `3 × 2`

`C = 6`, `НОК(3, 2) = 6`. Если время в бюджете — одометр идёт `(0, 0) (0, 1) (1, 0) (1, 1) (2, 0) (2, 1)`. Если время не влезло — прокрутка идёт `A+X → B+Y → C+X → A+Y → B+X → C+Y`. Множество проверенных выборок **одно и то же**, порядок разный (таблицы в разделах 11 и 12). Оба пути полны.

18.4. Подделка победила по подтверждениям

Сектор № 5: истина `A` пришла 2 раза, подделка `B` — 5 раз. Список `[B×5, A×2]`: лидер единоличный, головы нет → **точки ветвления нет**. Любая выборка берёт `B`, финальный хеш не сходится, сборка честно отказывает. Перебор бессилен: `A` вне пространства поиска (раздел 8).

Что делает система? Продолжает приём: повторы подтянут `A` до ничьей (`B×5, A×5`), и следующий вызов сборки уже увидит точку ветвления и начнёт ветвить. Ложного файла при этом не выдаётся никогда — в этом и состоит защита.

18.5. Тихо испорченный том

Файл из 64 data-томов и 8 ECC-томов. Том № 7 побит, но 72-битная проверка сектора прошла; версия одна, спорить не о чем. Стадии 1–2 принимают том «как есть» — хеш не сходится. Стадия 3 (раздел 13) исключает присутствующие тома по одному (уровень 1): на маске `{7}` RS восстанавливает истину из остальных, финальный хеш сходится. В худшем случае — несколько десятков дешёвых попыток, каждая с $E = 1$.

19. Настройки и пределы

Всё, что названо цифрами в этом пособии, — настройки объекта `SectorVersionSearchOptions`:

Настройка	Умолчание	Что ограничивает
<code>MaxExhaustiveCombinations</code>	100 000	потолок комбинаций для полного перебора
<code>MaxHeuristicAttempts</code>	100 000	потолок состояний прокрутки (включая исходное)
<code>TimeBudget</code>	30 с	мягкий бюджет; проверяется между попытками

Отдельная группа — свойство `SubsetSearch` для стадии 3 (раздел 13):

Настройка <code>SubsetSearch</code>	Умолчание	Что ограничивает
<code>MaxAttempts</code>	100 000	потолок попыток стадии
<code>TimeBudget</code>	30 с	мягкий бюджет стадии
<code>MaxErasedDataVolumes</code>	32	стёртых data-томов на одну попытку (кап E)
<code>ShortlistSize</code>	64	размер шорт-листа для пар и троек
<code>MaxExtraExclusionLevel</code>	3	максимальный размер доп. исключений

Правило большого пальца: поднимаете лимиты — растёт шанс достать истину из глубоких комбинаций, но растёт и худшее время отказа. Опускаете — интерфейс быстрее отвечает «не собирается», но чаще капитулирует раньше времени. Переполнения при подсчётах не стреляют исключениями: комбинации зажимаются в `long.MaxValue`, НОК — в лимит попыток; кап на E отдельно ограничивает цену одной попытки стадии 3.

20. Что гарантируется, а что нет

Гарантировано (доказуемо):

- Выдали файл → его SHA-256 равен `H3` → это исходник бит-в-бит (в рамках стойкости SHA-256).

2. Все data-тома приняты однозначно → успех с первой же попытки.
3. $c \leq 100\ 000$ и оценка времени в бюджете → правильная комбинация в головах будет найдена перебором (кроме отмены/бюджета — тогда честный отказ).
4. Размеры голов взаимно просты → прокрутка обходит все комбинации, эквивалент перебора.
5. Ложных результатов не бывает: либо доказанный файл, либо `null`.

Не гарантировано (эвристика):

6. При общих делителях размеров голов прокрутка видит только диагонали НОК.
7. Версии-меньшинства в текущем вызове не проверяются вовсе.
8. Укладка в 30 секунд — оценка по замеру нулевой попытки, а не обещание.
9. Тихо испорченный одиночный том вне коллизий будет восстановлен уровнем 1 стадии 3 — если хватает ЕСС-запаса и лимитов.
10. Пары и тройки тихих повреждений покрываются только в пределах шорт-листа и уровня исключений.

Цифры для памяти: хеш сектора — 72 бита, хеш заголовка — 192 бита, арбитр — SHA-256; перебор до 100 000 комбинаций, прокрутка до 100 000 состояний, подбор подмножества — до 100 000 масок при $E = 32$, бюджеты по 30 секунд.

21. Контрольные вопросы

1. Почему обычный повтор сектора — не коллизия, а укрепление версии? (раздел 4)
2. Из каких полей заголовка выводятся `N`, `M` и `T`? (разделы 1–2)
3. Почему результат сборки не может быть «частично правильным»? (раздел 3)
4. Почему версия с меньшим счётчиком не участвует в переборе, и как она может в него попасть? (разделы 8, 18.4)
5. По какому правилу выбирается между перебором и прокруткой? (раздел 10)
6. Почему для голов 3×2 прокрутка обходит всё, а для $2 \times 2 \times 2$ — только четверть? (раздел 11)
7. Что происходит с буферами при несовпадении финального хеша и зачем? (раздел 15)
8. Почему RS-восстановление выполняется после прямой сборки, а не до? (раздел 14)
9. Что увидит пользователь в прогрессе во время перебора? (раздел 16)
10. Может ли подделка стать собранным файлом? Почему? (разделы 3, 18.4)
11. Почему стадия подбора подмножества исключает целые тома, а не версии, и кого исключают первым? (раздел 13)